

Guía Completa de Instalación y Configuración

Chatbot RAG con Manual Empresarial - Windows 10

Documento: Guía de Instalación Completa

Fecha: Octubre 2025

Versión: 1.0

Sistema Operativo: Windows 10/11

Nivel: Principiante - Instalación desde cero

TABLA DE CONTENIDOS

PARTE 1: PREPARACIÓN DEL ENTORNO

- 1. [Introducción al Proyecto](#)
- 2. [Requisitos del Sistema](#)
- 3. [Arquitectura y Stack Tecnológico](#)

PARTE 2: INSTALACIÓN DE SOFTWARE BASE

- 4. [Instalación de Visual Studio Code](#)
- 5. [Instalación de Python](#)
- 6. [Instalación de Git \(Opcional\)](#)
- 7. [Verificación de Instalaciones](#)

PARTE 3: CONFIGURACIÓN DEL PROYECTO

- 8. [Creación de Estructura del Proyecto](#)
- 9. [Configuración de Entorno Virtual](#)
- 10. [Instalación de Dependencias Python](#)
- 11. [Configuración de API Keys](#)

PARTE 4: VERIFICACIÓN Y PRUEBAS

- 12. [Primera Ejecución y Tests](#)
- 13. [Solución de Problemas Comunes](#)

PARTE 5: PRÓXIMOS PASOS

- 14. [Implementación del Sistema RAG](#)
- 15. [Roadmap de Desarrollo](#)

ANEXOS

- [Anexo A: Comandos de Referencia Rápida](#)

- [Anexo B: Glosario de Términos](#)
- [Anexo C: Recursos y Enlaces](#)

PARTE 1: PREPARACIÓN DEL ENTORNO

1. Introducción al Proyecto {#1-introducción}

1.1 ¿Qué vamos a construir?

Un **chatbot inteligente** que puede responder preguntas basándose en el manual de tu empresa utilizando tecnología RAG (Retrieval-Augmented Generation).

1.2 ¿Qué es RAG?

RAG = Retrieval-Augmented Generation

Es una técnica que combina:

- 🔍 **Retrieval (Recuperación)**: Buscar información relevante en documentos
- 🧠 **Augmented (Aumentado)**: Enriquecer las respuestas con ese contexto
- ✍️ **Generation (Generación)**: Crear respuestas naturales usando IA

Ventajas para tu caso:

- ✅ Se actualiza fácilmente (manual mensual)
- ✅ No requiere entrenamiento costoso
- ✅ Cita fuentes específicas del manual
- ✅ Respuestas precisas y actualizadas
- ✅ Implementación en semanas (no meses)

1.3 Flujo de Funcionamiento

```

Usuario: "¿Cuál es el proceso de solicitud de vacaciones?"
↓
1. Sistema busca en el manual: "vacaciones", "solicitud", "proceso"
↓
2. Encuentra sección relevante (Capítulo 5.2)
↓
3. LLM lee esa sección y genera respuesta natural
↓
4. Usuario recibe: "Según el manual (Cap. 5.2), el proceso es..."
    
```

1.4 Casos de Uso

- 📖 Soporte interno a empleados
- 🏢 Onboarding de nuevos empleados

- 📞 Reducción de consultas a RRHH
- 🕒 Acceso 24/7 a información del manual
- 📊 Analytics de preguntas frecuentes

2. Requisitos del Sistema {#2-requisitos}

2.1 Requisitos de Hardware

Mínimos (Desarrollo Básico)

- **Procesador:** Intel Core i5 (6ta gen) / AMD Ryzen 5
- **RAM:** 8 GB
- **Disco Duro:** 10 GB libres
- **Conexión:** Internet banda ancha

Recomendados (Desarrollo Óptimo)

- **Procesador:** Intel Core i7 / AMD Ryzen 7
- **RAM:** 16 GB
- **Disco:** SSD con 20 GB libres
- **GPU:** Opcional (acelera embeddings locales)

2.2 Requisitos de Software

- **Sistema Operativo:** Windows 10/11 (64-bit)
- **Navegador:** Chrome, Firefox o Edge actualizado
- **Permisos:** Acceso de administrador para instalaciones

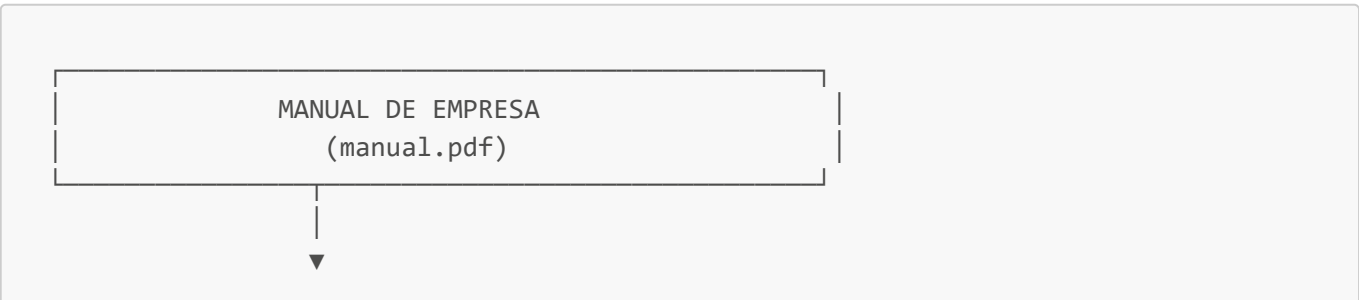
2.3 Cuentas Necesarias

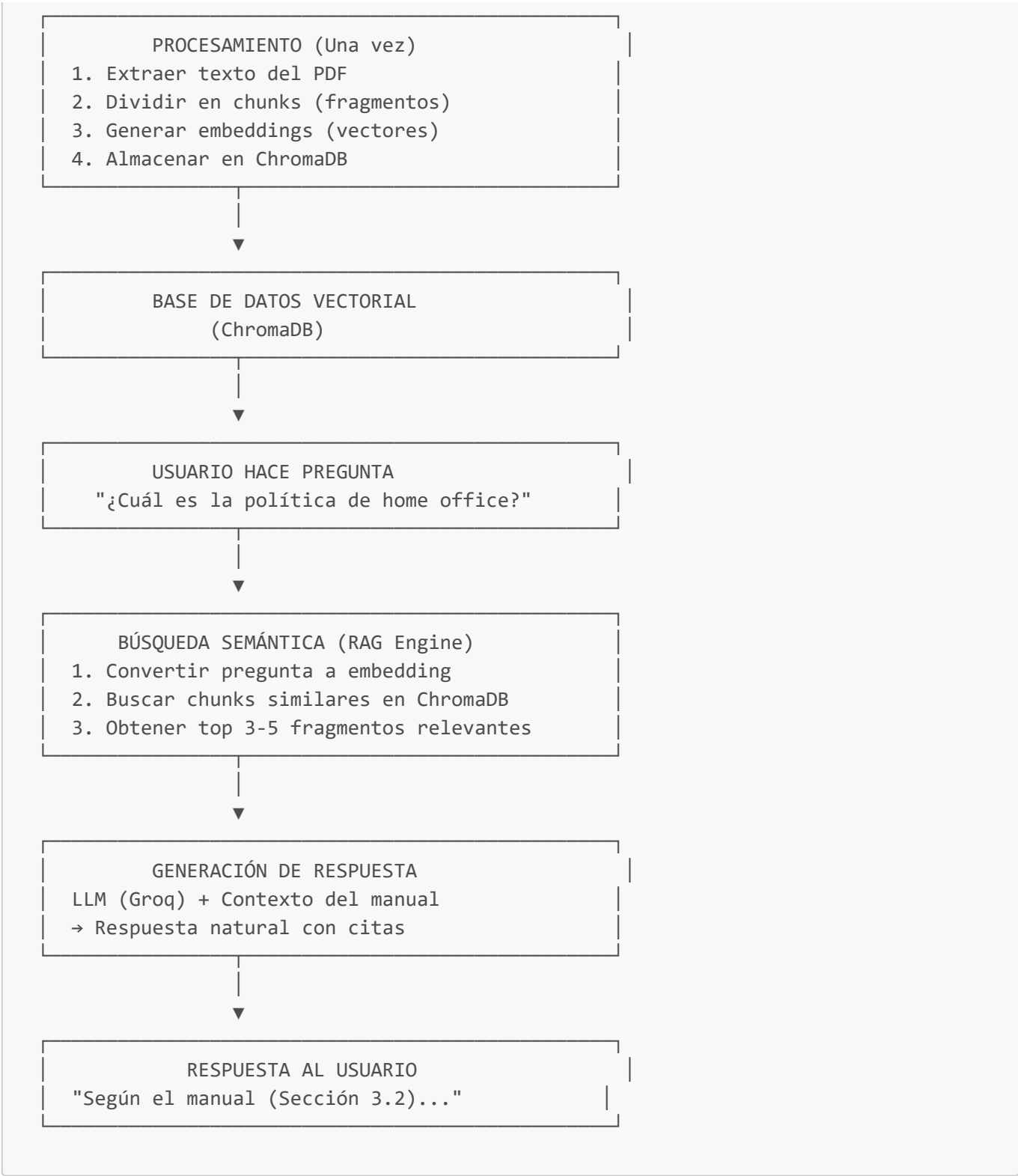
Servicio	Coste	Uso	Límite Gratuito
Groq	Gratis	LLM (IA)	100K tokens/día
GitHub	Gratis	Control versiones	Ilimitado

Total invertido: \$0.00

3. Arquitectura y Stack Tecnológico {#3-arquitectura}

3.1 Diagrama de Arquitectura





3.2 Stack Tecnológico Detallado

Frontend (Futuro)

- **Next.js 14:** Framework React
- **Tailwind CSS:** Estilos
- **shadcn/ui:** Componentes UI

Backend

- **FastAPI:** Framework web Python

- **Uvicorn:** Servidor ASGI
- **Python 3.11+:** Lenguaje base

IA y RAG

Componente	Tecnología	Función
LLM	Groq (Llama 3.3)	Generación de respuestas
Embeddings	Sentence-Transformers	Vectorización de texto
Vector DB	ChromaDB	Almacenamiento y búsqueda
Orquestación	LangChain	Framework RAG
PDF Processing	PyPDF	Extracción de texto

Infraestructura

- **Local Development:** Todo en tu PC
- **Production (futuro):** Railway, Vercel o AWS

3.3 Flujo de Datos Detallado

Fase 1: Procesamiento del Manual (Una vez)

PDF → PyPDF → Texto plano

Texto → Chunking (500 tokens/chunk) → Chunks

Chunks → SentenceTransformer → Embeddings (vectores 384D)

Embeddings → ChromaDB → Almacenamiento persistente

Fase 2: Consulta del Usuario (Cada pregunta)

Pregunta usuario → Embedding (vector 384D)

Vector pregunta + ChromaDB → Búsqueda similaridad coseno

Top 5 chunks → Contexto

Contexto + Pregunta + Prompt → Groq LLM

LLM → Respuesta natural → Usuario

PARTE 2: INSTALACIÓN DE SOFTWARE BASE

4. Instalación de Visual Studio Code {#4-vscode}

4.1 ¿Qué es VS Code?

Visual Studio Code es un editor de código gratuito y ligero creado por Microsoft. Lo usaremos para:

- Escribir código Python
- Ejecutar scripts
- Gestionar archivos del proyecto
- Depurar errores

4.2 Descarga

1. Abrir navegador web
2. Ir a: **<https://code.visualstudio.com/>**
3. Click en **"Download for Windows"** (botón azul grande)
4. Esperar descarga (≈80 MB, 1-2 minutos)

4.3 Instalación Paso a Paso

Paso 1: Localizar archivo descargado

- Nombre: **VSCodeUserSetup-x64-1.XX.X.exe**
- Ubicación típica: **C:\Users\TU_USUARIO\Downloads**

Paso 2: Ejecutar instalador

- **Doble click** en el archivo **.exe**
- Si aparece advertencia de seguridad → Click **"Sí"**

Paso 3: Pantalla de bienvenida

- Click **"Siguiente"** (Next)

Paso 4: Aceptar licencia

- Leer términos
- Seleccionar **"Acepto el acuerdo"**
- Click **"Siguiente"**

Paso 5: Ubicación de instalación

- Dejar por defecto: **C:\Users\TU_USUARIO\AppData\Local\Programs\Microsoft VS Code**
- Click **"Siguiente"**

Paso 6: Carpeta del menú inicio

- Dejar por defecto: "Visual Studio Code"
- Click **"Siguiente"**

Paso 7: ⚠ **OPCIONES IMPORTANTES** ⚠ Marcar TODAS estas casillas:

- ☒ **"Crear un icono en el escritorio"**
- ☒ **"Agregar la acción 'Abrir con Code' al menú contextual"**
- ☒ **"Agregar la acción 'Abrir con Code' al menú contextual de directorios"**
- ☒ **"Registrar Code como editor para tipos de archivo admitidos"**

- ☒ **"Agregar a PATH (disponible después de reiniciar)"** ← MUY IMPORTANTE

Paso 8: Instalar

- Click **"Instalar"**
- Esperar 2-3 minutos (barra de progreso)

Paso 9: Finalizar

- ☒ Marcar **"Ejecutar Visual Studio Code"**
- Click **"Finalizar"**

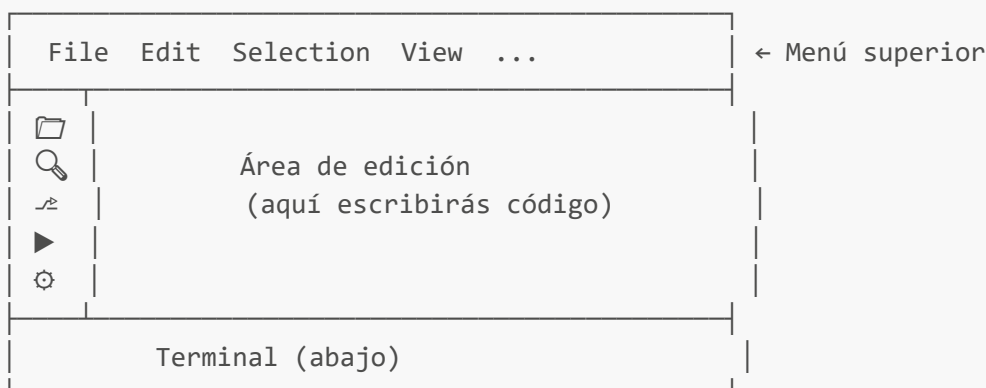
4.4 Primera Ejecución de VS Code

VS Code se abrirá automáticamente. Verás:

Pantalla de bienvenida:

- Selector de tema (claro/oscuro)
- Recomendación: Elegir **"Dark+"** (menos cansado para la vista)

Interfaz principal:



4.5 Instalación de Extensiones Esenciales

Paso 1: Abrir panel de extensiones

- Click en icono de cuadrados (lado izquierdo)
- O presionar: **Ctrl + Shift + X**

Paso 2: Instalar extensión de Python

1. En buscador escribir: **"Python"**
2. Buscar: **"Python" by Microsoft** (icono azul)
3. Click **"Install"**
4. Esperar 30 segundos

Paso 3: Instalar Pylance

1. Buscar: **"Pylance"**
2. Seleccionar: **"Pylance" by Microsoft**
3. Click **"Install"**
4. Esperar 20 segundos

Extensiones Opcionales (Recomendadas):

Extensión	Descripción	Necesaria
Python Indent	Indentación automática	Recomendada
autoDocstring	Documentación automática	Opcional
Error Lens	Muestra errores inline	Recomendada
Markdown All in One	Para documentación	Opcional

4.6 Solución de Problemas VS Code

Problema 1: Error 5 - No hay permisos

Síntoma: Al abrir VS Code aparece "Error 5: Acceso denegado"

Causa: Falta de permisos de administrador

Solución:

1. Cerrar VS Code completamente
2. Buscar icono de VS Code (escritorio o menú inicio)
3. **Click derecho** → **"Ejecutar como administrador"**
4. Click **"Sí"** en ventana UAC

Solución permanente:

1. Click derecho en icono VS Code
2. **"Propiedades"**
3. Pestaña **"Compatibilidad"**
4. ☒ Marcar **"Ejecutar como administrador"**
5. **"Aplicar"** → **"Aceptar"**

Problema 2: VS Code no se encuentra en PATH

Síntoma: Al escribir `code` en CMD no se reconoce

Solución: Reinstalar marcando opción "Agregar a PATH"

5. Instalación de Python {#5-python}

5.1 ¿Qué es Python?

Python es el lenguaje de programación que usaremos. Es:

- 📖 Fácil de aprender
- 📦 Tiene librerías para IA (LangChain, transformers)
- 🌐 Usado por millones de desarrolladores

Versión requerida: Python 3.11 o superior

5.2 Descarga

1. Abrir navegador
2. Ir a: <https://www.python.org/downloads/>
3. Click en "**Download Python 3.11.X**" (botón amarillo)
 - Nota: X es el número de versión minor (ej: 3.11.5)
4. Esperar descarga (≈25 MB, 1 minuto)

5.3 Instalación Paso a Paso

⚠ PASO CRÍTICO ⚠

Paso 1: Ejecutar instalador

- Localizar: `python-3.11.X-amd64.exe` en Descargas
- **Doble click**

Paso 2: ⚠ PANTALLA INICIAL - MUY IMPORTANTE ⚠

Verás dos opciones abajo:

- ☒ **"Add python.exe to PATH"** ← **MARCAR ESTO OBLIGATORIAMENTE**
- ☐ "Install launcher for all users"

SI NO MARCAS "Add to PATH", Python no funcionará correctamente

Paso 3: Modo de instalación

- Click **"Install Now"** (opción recomendada)
- Aparecerá solicitud de permisos UAC → Click **"Sí"**

Paso 4: Progreso

- Barra de progreso: "Installing..."
- Esperar 2-3 minutos

Paso 5: Finalización

- Verás "Setup was successful"
- Opcional: Click **"Disable path length limit"** (recomendado)
- Click **"Close"**

5.4 Verificación de Instalación

Método 1: CMD

1. Presionar `Windows + R`

2. Escribir: `cmd`
3. Presionar `Enter`
4. En ventana negra escribir:

```
python --version
```

5. Presionar `Enter`

Resultado esperado:

```
Python 3.11.5
```

Si aparece error:

```
'python' no se reconoce como un comando interno o externo...
```

→ Python NO está en PATH. Reinstalar marcando la casilla.

Método 2: Verificar pip (gestor de paquetes)

En misma ventana CMD:

```
pip --version
```

Resultado esperado:

```
pip 23.1.2 from C:\Users\...\site-packages\pip (python 3.11)
```

5.5 Prueba de Python Interactivo

1. En CMD escribir:

```
python
```

2. Verás:

```
Python 3.11.5 (...)  
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```

3. Escribir:

```
print("¡Python funciona!")
```

4. Presionar **Enter**

Resultado esperado:

```
¡Python funciona!
```

5. Para salir:

```
exit()
```

5.6 Actualizar pip (Recomendado)

```
python -m pip install --upgrade pip
```

Esperar 30 segundos. Verás:

```
Successfully installed pip-24.X.X
```

5.7 Solución de Problemas Python

Problema 1: Python no está en PATH

Síntoma: `'python'` no se reconoce como comando

Solución A - Reinstalar correctamente:

1. Panel de Control → Programas → Desinstalar
2. Buscar "Python 3.11"
3. Desinstalar
4. Reiniciar PC
5. Reinstalar marcando "Add to PATH"

Solución B - Añadir manualmente a PATH:

1. Buscar en Windows: "Variables de entorno"
2. Click "Variables de entorno del sistema"
3. En "Variables del sistema" → buscar "Path"

4. Click "Editar"
5. Click "Nuevo"
6. Añadir: `C:\Users\TU_USUARIO\AppData\Local\Programs\Python\Python311`
7. Añadir: `C:\Users\TU_USUARIO\AppData\Local\Programs\Python\Python311\Scripts`
8. "Aceptar" todo
9. Cerrar y reabrir CMD

Problema 2: Múltiples versiones de Python

Síntoma: `python --version` muestra versión antigua

Solución:

- Desinstalar todas las versiones antiguas
- Instalar solo Python 3.11+

Problema 3: pip no funciona

Síntoma: `'pip'` no se reconoce como comando

Solución:

```
python -m ensurepip --upgrade
python -m pip install --upgrade pip
```

6. Instalación de Git (Opcional pero Recomendado) {#6-git}

6.1 ¿Qué es Git y por qué instalarlo?

Git es un sistema de control de versiones. Te permite:

- 📷 Guardar "fotografías" del código en cada cambio
- 🔄 Volver a versiones anteriores si algo falla
- 🌿 Trabajar en ramas (branches) sin afectar código principal
- 🤝 Colaborar con otros desarrolladores (futuro)

¿Es obligatorio ahora? No, pero facilita el desarrollo.

6.2 Descarga

1. Ir a: <https://git-scm.com/download/win>
2. Descarga automática empezará (≈50 MB)
3. Si no empieza: Click **"Click here to download manually"**

6.3 Instalación

Paso 1: Ejecutar instalador

- Archivo: **Git-2.XX.X-64-bit.exe**
- Doble click

Paso 2-8: Opciones de instalación

- Todas las pantallas: Click **"Next"** (opciones por defecto están bien)

Paso 9: Editor por defecto

- Seleccionar: **"Use Visual Studio Code as Git's default editor"**
- Click "Next"

Paso 10-15: Más opciones

- Dejar todo por defecto
- Click "Next" en todas

Paso 16: Instalar

- Click **"Install"**
- Esperar 2 minutos

Paso 17: Finalizar

- Click **"Finish"**

6.4 Verificación de Git

Abrir CMD y ejecutar:

```
git --version
```

Resultado esperado:

```
git version 2.42.0.windows.1
```

6.5 Configuración Inicial de Git

Ejecutar estos comandos (reemplazar con tus datos):

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tu@email.com"
```

Verificar configuración:

```
git config --list
```

7. Verificación de Instalaciones {#7-verificacion}

7.1 Checklist de Verificación

Ejecutar cada comando en CMD (Windows + R → cmd):

Software	Comando	Resultado Esperado
Python	<code>python --version</code>	Python 3.11.X
pip	<code>pip --version</code>	pip 23.X.X
Git	<code>git --version</code>	git version 2.X.X
VS Code	<code>code --version</code>	1.XX.X

7.2 Script de Verificación Automática

Crear archivo `verificar_instalacion.bat`:

```
@echo off
echo =====
echo VERIFICACION DE INSTALACIONES
echo =====
echo.

echo Verificando Python...
python --version
if %errorlevel% neq 0 (
    echo [ERROR] Python no encontrado
) else (
    echo [OK] Python instalado
)
echo.

echo Verificando pip...
pip --version
if %errorlevel% neq 0 (
    echo [ERROR] pip no encontrado
) else (
    echo [OK] pip instalado
)
echo.

echo Verificando Git...
git --version
if %errorlevel% neq 0 (
    echo [ADVERTENCIA] Git no encontrado (opcional)
) else (
    echo [OK] Git instalado
)
)
```

```
echo.  
  
echo Verificando VS Code...  
code --version  
if %errorlevel% neq 0 (  
    echo [ERROR] VS Code no encontrado en PATH  
) else (  
    echo [OK] VS Code instalado  
)  
echo.  
  
echo =====  
echo VERIFICACION COMPLETADA  
echo =====  
pause
```

Ejecutar: Doble click en el archivo **.bat**

PARTE 3: CONFIGURACIÓN DEL PROYECTO

8. Creación de Estructura del Proyecto {#8-estructura}

8.1 Crear Carpeta del Proyecto

Método 1: Explorador de Windows

1. Abrir **Explorador de Archivos** (Windows + E)
2. Navegar a: **C:\Users\TU_USUARIO\Documents**
3. Click derecho → **"Nuevo"** → **"Carpeta"**
4. Nombrar: **Proyectos**
5. Entrar a la carpeta **Proyectos**
6. Click derecho → **"Nuevo"** → **"Carpeta"**
7. Nombrar: **chatbot-manual**

Ruta final: **C:\Users\TU_USUARIO\Documents\Proyectos\chatbot-manual**

Método 2: CMD (Alternativo)

```
cd C:\Users\%USERNAME%\Documents  
mkdir Proyectos  
cd Proyectos  
mkdir chatbot-manual  
cd chatbot-manual
```

8.2 Abrir Proyecto en VS Code

Método 1: Desde VS Code

1. Abrir VS Code
2. Menú: **File** → **Open Folder**
3. Navegar a: `C:\Users\TU_USUARIO\Documents\Proyectos\chatbot-manual`
4. Click **"Seleccionar carpeta"**
5. Si pregunta "Do you trust the authors?": Click **"Yes, I trust the authors"**

Método 2: Click derecho (si instalaste VS Code correctamente)

1. Ir a carpeta en Explorador
2. Click derecho dentro de la carpeta (espacio vacío)
3. **"Abrir con Code"**

8.3 Estructura Inicial del Proyecto

Dentro de VS Code, crear esta estructura:

```
chatbot-manual/
├── README.md           # Documentación del proyecto
├── .gitignore          # Archivos a ignorar en Git
├── .env                # Variables de entorno (API keys)
├── requirements.txt     # Dependencias Python
├── data/               # Carpeta para datos
│   └── manual.pdf      # Aquí irá el manual
├── src/                # Código fuente
│   ├── __init__.py
│   ├── process_manual.py # Procesar PDF
│   ├── rag_engine.py    # Motor RAG
│   └── chatbot.py       # Interfaz chat
├── tests/              # Tests
│   └── test_basic.py
└── docs/               # Documentación extra
    └── guia_uso.md
```

8.4 Crear Archivos Iniciales

Paso 1: Crear README.md

1. En VS Code, click derecho en explorador → **"New File"**
2. Nombrar: `README.md`
3. Contenido inicial:

```
# Chatbot RAG - Manual Empresarial
```


Sistema de chatbot inteligente basado en RAG para consultar el manual de la empresa.

Estado

 En desarrollo

Tecnologías

- Python 3.11+
- LangChain
- ChromaDB
- Groq API (Llama 3.3)

Instalación

Ver ``docs/INSTALACION.md``

Paso 2: Crear .gitignore

Contenido:

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
venv/
env/
.env

# IDEs
.vscode/
.idea/

# Datos
data/*.pdf
*.db
chroma_db/

# Logs
*.log

# OS
.DS_Store
Thumbs.db
```

Paso 3: Crear carpetas

En VS Code:

- Click derecho → **"New Folder"** → Nombrar: **data**
- Click derecho → **"New Folder"** → Nombrar: **src**

- Click derecho → "**New Folder**" → Nombrar: **tests**
 - Click derecho → "**New Folder**" → Nombrar: **docs**
-

9. Configuración de Entorno Virtual {#9-entorno}

9.1 ¿Qué es un Entorno Virtual?

Un **entorno virtual** es una carpeta aislada que contiene:

- Una copia de Python
- Las librerías específicas del proyecto
- Configuración independiente

Ventajas:

- ☒ No contamina el Python del sistema
- ☒ Evita conflictos entre proyectos
- ☒ Fácil de borrar y recrear
- ☒ Portabilidad (compartir con equipo)

9.2 Abrir Terminal en VS Code

1. En VS Code, menú: **Terminal** → **New Terminal**
2. O atajo: **Ctrl + Ñ** (o **Ctrl + `**)
3. Aparecerá panel abajo

Verificar ruta:

```
PS C:\Users\TU_USUARIO\Documents\Proyectos\chatbot-manual>
```

9.3 Crear Entorno Virtual

En la terminal de VS Code, ejecutar:

```
python -m venv venv
```

Explicación del comando:

- **python**: Ejecuta Python
- **-m venv**: Usa el módulo **venv** (virtual environment)
- **venv**: Nombre de la carpeta (convención: llamarla "venv")

Tiempo: 20-30 segundos

Resultado: Se crea carpeta **venv/** con:

```
venv/
├── Include/
├── Lib/
│   └── site-packages/ ← Aquí van las librerías
├── Scripts/
│   ├── activate.bat ← Script de activación (CMD)
│   ├── Activate.ps1 ← Script de activación (PowerShell)
│   └── python.exe ← Python del entorno
└── pyvenv.cfg
```

9.4 Activar Entorno Virtual

Identificar tu terminal:

En VS Code, mira la terminal. Verás:

- **PS** = PowerShell
- **C:\Users\...** (sin PS) = CMD

Si es PowerShell:

```
venv\Scripts\activate
```

Si es CMD:

```
venv\Scripts\activate.bat
```

Verificación exitosa: Verás **(venv)** al inicio:

```
(venv) PS C:\Users\adria\Documents\Proyectos\chatbot-manual>
```

9.5 Solución: Error PowerShell (Scripts Deshabilitados)

Error común:

No se puede cargar el archivo... la ejecución de scripts está deshabilitada

Solución rápida:

1. Abrir **PowerShell como Administrador**:

- Buscar "PowerShell" en Windows
- Click derecho → "Ejecutar como administrador"

2. Ejecutar:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

- 3. Escribir `S` y Enter
- 4. Cerrar PowerShell
- 5. Volver a VS Code y ejecutar `venv\Scripts\activate`

Alternativa: Cambiar a CMD

- En terminal VS Code: Click flecha abajo → "Command Prompt"
- Usar: `venv\Scripts\activate.bat`

10. Instalación de Dependencias Python {#10-dependencias}

10.1 Instalar Librerías (5-10 minutos)

Con el entorno virtual activado (`venv`), ejecutar:

```
pip install langchain langchain-community chromadb sentence-transformers pypdf python-dotenv fastapi uvicorn groq
```

Luego instalar módulo específico de Groq:

```
pip install langchain-groq
```

Tiempo total: 5-10 minutos (ChromaDB y sentence-transformers son pesados)

Qué instalan estas librerías:

Librería	Función
langchain	Framework RAG principal
langchain-community	Integraciones extra
langchain-groq	Conexión con Groq API
chromadb	Base de datos vectorial
sentence-transformers	Crear embeddings
pypdf	Leer archivos PDF
python-dotenv	Variables de entorno
fastapi	API web (futuro)

Librería	Función
uvicorn	Servidor web (futuro)
groq	Cliente API Groq

10.2 Verificar Instalación

```
pip list
```

Deberías ver todas las librerías instaladas.

10.3 Crear requirements.txt

```
pip freeze > requirements.txt
```

Esto guarda todas las dependencias en un archivo para futuras instalaciones.

11. Configuración de API Keys {#11-apikeys}

11.1 Obtener API Key de Groq

Paso 1: Ir a <https://console.groq.com/>

Paso 2: Registrarse

- Click "Sign in"
- Registrar con Google o email
- Es 100% gratis

Paso 3: Crear API Key

- Menú izquierdo: "API Keys"
- Click "Create API Key"
- Darle un nombre: "chatbot-manual"
- Click "Submit"

Paso 4: Copiar la key

- Aparecerá algo como: `gsk_abc123...`
- **Copiarla completa**
- **Guardarla** (no se puede recuperar después)

11.2 Crear archivo .env

En VS Code, raíz del proyecto:

1. Click derecho → "New File"
2. Nombrar: `.env` (con el punto al inicio)
3. Contenido:

```
GROQ_API_KEY=gsk_tu_key_aqui_pegala_completa
```

4. Guardar: `Ctrl + S`

⚠ **Importante:** NO subir este archivo a Git (ya está en `.gitignore`)

12. Primera Ejecución y Tests {#12-tests}

12.1 Test de Conexión con Groq

Crear archivo `test.py` en la raíz:

```
from langchain_groq import ChatGroq
from dotenv import load_dotenv
import os

load_dotenv()

print("🔗 Probando conexión con Groq...")

try:
    llm = ChatGroq(
        temperature=0,
        model_name="llama-3.3-70b-versatile",
        groq_api_key=os.getenv("GROQ_API_KEY")
    )

    response = llm.invoke("Di: '¡Todo funciona correctamente!'")
    print("\n✅ ÉXITO:")
    print(response.content)

except Exception as e:
    print(f"\n❌ ERROR: {e}")
```

12.2 Ejecutar Test

En terminal (con `venv` activado):

```
python test.py
```

Resultado esperado:

🔧 Probando conexión con Groq...

✅ ÉXITO:
¡Todo funciona correctamente!

12.3 Solución de Errores

Error: "model_decommissioned"

Cambiar modelo en test.py:

```
model_name="llama-3.3-70b-versatile" # Modelo actualizado
```

Modelos disponibles:

- llama-3.3-70b-versatile (recomendado)
- llama-3.1-8b-instant (más rápido)
- mixtral-8x7b-32768

Error: "No module named 'langchain_groq'"

```
pip install langchain-groq
```

Error: "Invalid API Key"

- Verificar que .env existe
- Verificar que la key está completa
- Sin espacios extras

13. Solución de Problemas Comunes {#13-problemas}

Problema: Python no se reconoce

Solución: Reinstalar Python marcando "Add to PATH"

Problema: Scripts PowerShell deshabilitados

Solución:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Problema: pip install muy lento

Solución: Es normal, tarda 5-10 minutos en primera instalación

Problema: Entorno virtual no se activa

PowerShell:

```
venv\Scripts\activate
```

CMD:

```
venv\Scripts\activate.bat
```

Problema: Error al importar módulos

Solución: Verificar que entorno virtual está activado (debe aparecer **(venv)**)

14. Implementación del Sistema RAG {#14-rag}

14.1 Próximos Archivos a Crear

```
src/  
├─ process_manual.py    # Procesar PDF → ChromaDB  
├─ rag_engine.py        # Motor de búsqueda  
└─ chatbot.py           # Interfaz de usuario
```

14.2 Flujo de Desarrollo

Fase 1: Procesar manual (process_manual.py)

- Leer PDF
- Dividir en chunks
- Crear embeddings
- Guardar en ChromaDB

Fase 2: Motor RAG (rag_engine.py)

- Búsqueda semántica
- Recuperar contexto relevante
- Generar respuesta con LLM

Fase 3: Chatbot (chatbot.py)

- Interfaz CLI simple
- Conversación interactiva
- Historial de contexto

15. Roadmap de Desarrollo {#15-roadmap}

Semana 1: Core RAG

-  Setup completo
-  Procesador de PDF
-  Motor RAG básico
-  Chatbot CLI

Semana 2: Mejoras

-  Citas de fuente
-  Detección fuera de contexto
-  Optimización chunks

Semana 3: API

-  FastAPI backend
-  Endpoints REST
-  Documentación Swagger

Semana 4: Frontend

-  Interfaz web React
-  Chat UI
-  Deploy

ANEXO A: Comandos de Referencia Rápida {#anexo-a}

Comandos Windows

```
# Abrir CMD
Windows + R → cmd

# Navegar carpetas
cd C:\ruta\carpeta
cd .. # Subir nivel

# Listar archivos
dir

# Crear carpeta
mkdir nombre_carpeta
```

Comandos Python

```
# Versión Python
python --version

# Versión pip
pip --version

# Crear entorno virtual
python -m venv venv

# Activar entorno (PowerShell)
venv\Scripts\activate

# Activar entorno (CMD)
venv\Scripts\activate.bat

# Instalar librería
pip install nombre_libreria

# Ver librerías instaladas
pip list

# Guardar dependencias
pip freeze > requirements.txt

# Instalar desde requirements
pip install -r requirements.txt

# Ejecutar script
python script.py
```

Comandos VS Code

```
Ctrl + Ñ      → Abrir/cerrar terminal
Ctrl + Shift + P → Paleta de comandos
Ctrl + P      → Buscar archivo
Ctrl + S      → Guardar
Ctrl + Shift + X → Extensiones
Ctrl + `      → Terminal alternativo
```

ANEXO B: Glosario de Términos {#anexo-b}

API (Application Programming Interface): Interfaz para que programas se comuniquen

ChromaDB: Base de datos especializada en vectores para búsqueda semántica

Chunking: Dividir texto en fragmentos pequeños

CLI (Command Line Interface): Interfaz de línea de comandos

Embedding: Representación numérica (vector) de texto

LangChain: Framework para aplicaciones con LLMs

LLM (Large Language Model): Modelo de IA grande (como GPT, Llama)

PATH: Variable que indica dónde buscar programas ejecutables

pip: Gestor de paquetes de Python

RAG (Retrieval-Augmented Generation): Técnica que combina búsqueda + generación

Vector Database: Base de datos optimizada para buscar vectores similares

Entorno Virtual: Instalación aislada de Python con sus propias librerías

ANEXO C: Recursos y Enlaces {#anexo-c}

Descargas

- Python: <https://www.python.org/downloads/>
- VS Code: <https://code.visualstudio.com/>
- Git: <https://git-scm.com/download/win>

APIs y Servicios

- Groq Console: <https://console.groq.com/>
- Groq Docs: <https://console.groq.com/docs/quickstart>

Documentación

- LangChain: <https://python.langchain.com/>
- ChromaDB: <https://docs.trychroma.com/>
- FastAPI: <https://fastapi.tiangolo.com/>

Comunidades

- LangChain Discord: <https://discord.gg/langchain>
 - Groq Discord: <https://groq.com/discord>
-

Checklist Final

Antes de continuar, verifica que tienes:

- ☐ Windows 10/11 64-bit
- ☐ VS Code instalado y funcionando
- ☐ Python 3.11+ instalado (`python --version`)
- ☐ pip funcionando (`pip --version`)
- ☐ Carpeta proyecto creada
- ☐ Proyecto abierto en VS Code

- ☐ Entorno virtual creado (`venv/` existe)
- ☐ Entorno virtual activado (aparece (`venv`))
- ☐ Todas las librerías instaladas (sin errores)
- ☐ Archivo `.env` creado con API key de Groq
- ☐ Test ejecutado con éxito (`python test.py`)

Si todo está marcado → ¡Listo para implementar el sistema RAG!

Documento: Guía de Instalación Completa

Versión: 1.0

Fecha: Octubre 2025

Estado: Setup base completado ☒